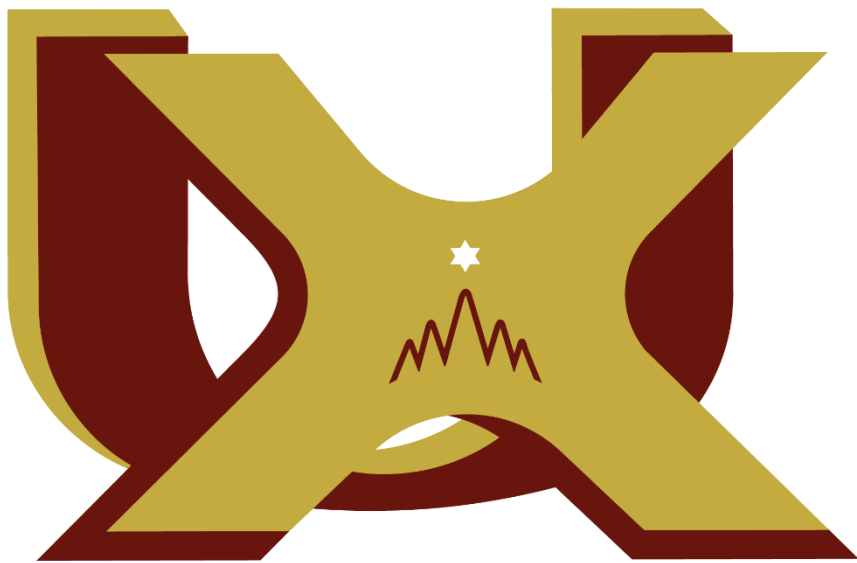


TORRE DE HANÓI

Héctor Luis Garcia Marthen

Universidad Xalapa



UNIVERSIDAD DE XALAPA

• ————— •
Saber / Trascender

ORIGEN Y CREADOR

- **Creador:** El matemático francés **Édouard Lucas**.
- **Año:** Fue comercializado por primera vez como juego en **1883** bajo el seudónimo de "N. Claus de Siam" (un anagrama de Lucas d'Amiens).

Objetivo y Reglas

- **Objetivo:** Trasladar una pila de n discos colocados de mayor a menor tamaño desde una torre de origen hasta una torre destino, utilizando una torre auxiliar.
- **Reglas:**
 1. Solo se puede mover un disco a la vez.
 2. Un movimiento consiste en tomar el disco superior de una de las torres y colocarlo encima de otra torre o sobre una torre vacía.
 3. No se puede colocar un disco más grande encima de uno más pequeño.

Conceptos de Recursividad y Ciencias de la Computación

- **¿Por qué es un ejemplo clásico de recursividad?:** Porque la solución para n discos depende directamente de resolver el mismo problema exacto pero para $n - 1$ discos.

- **Aplicaciones reales y computacionales:** Se utiliza en la enseñanza de diseño de algoritmos (divide y vencerás), pruebas de rendimiento de procesadores, gestión de memoria en sistemas operativos (pila de llamadas), algoritmos de búsqueda y planificación, y en respaldos de datos basados en cintas (estrategia de rotación de Hanói).
- **Número de llamadas recursivas:** Un problema de n discos genera exactamente $2^n - 1$ **llamadas recursivas**.
- **Relación con la pila de llamadas (Call Stack):** Sigue la estructura **LIFO** (*Last In, First Out*). El algoritmo apila llamadas suspendidas a medida que desciende hacia el caso base ($n = 1$) y las desapila al retornar, mapeando el estado exacto de cada submoviedura en la memoria.
- **Crecimiento acelerado de movimientos:** Es un problema de complejidad exponencial $\mathcal{O}(2^n)$. El número de movimientos mínimos requeridos se calcula con la fórmula $M(n) = 2^n - 1$. Cada disco adicional duplica el esfuerzo acumulado previo más uno.
- **Ventajas de la solución recursiva:** Código extremadamente elegante, corto, declarativo y fácil de verificar matemáticamente en comparación con una implementación iterativa basada en pilas explícitas.

- **Desventajas para un n grande:**
 1. **Desbordamiento de pila (*Stack Overflow*):** Consumo excesivo de memoria en la pila al acumular llamadas sin retornar.
 2. **Tiempo de ejecución impráctico:** Crece exponencialmente; procesar valores grandes tardaría miles o millones de años.

Referencias (Formato APA)

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
- Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C++* (4th ed.). Pearson.

Análisis y Respuestas a las Preguntas de Ejecución

- **¿Cuántos movimientos se requieren para 30 discos?**

$$2^{30} - 1 = 1,073,741,823 \text{ movimientos (aprox. } 1.07 \times 10^9)$$

- **¿Cuántos movimientos se requieren para 40 discos?**

$$2^{40} - 1 = 1,099,511,627,775 \text{ movimientos (aprox. } 1.1 \times 10^{12})$$

- **¿Sería práctico realizar físicamente los movimientos para 40 discos?**

No, es completamente inviable. Si una persona hiciera 1 movimiento por segundo sin descansar ni cometer errores, tomaría **más de 34,865 años** completar los 1,099,511,627,775 movimientos.

- **¿Qué sucede con el número de movimientos cuando n aumenta?**

Crece de forma **exponencial**. Cada vez que se añade un único disco ($n + 1$), el tiempo y la cantidad de operaciones requeridas se **duplican más un paso extra**.

- **¿Qué relación observas entre este crecimiento y la eficiencia de un algoritmo?**

Los algoritmos con complejidad exponencial $\mathcal{O}(2^n)$ son **ineficientes e intratables** para entradas medianas o grandes. Demuestran que la

elegancia de una solución no garantiza su aplicabilidad práctica cuando la escala aumenta.